

RIX 2024 - Batch 1- NUnit & Jest Practical Assessment Assessment

12576 Nisanth Saravanan

Online Courier Management System

An online courier management system is a software application that is used to manage the process of sending and tracking packages and documents through a courier service. This typically includes functions such as creating shipping labels, tracking packages, and generating reports. This project is a useful tool for businesses and individuals who need to send packages and documents through a courier service on a regular basis.

W3H Analysis

Project Title: Online Courier Management System	
What?	How?
1. What are the necessary modules? A: Modules Needed: 1) Login and Registration. 2) Admin Dashboard. 3) User Dashboard.	Login and Registration Module: 1) Register User: Method 1: Use Simple Form Page. Method 2: Use a Modular Form Page. Method 3: Use Google OAuth and other 3rd party services.
2. What are the necessary functionalities of Login and Registration Modules? A: Login and Registration Module: 1) Register User. 2) Login as User. 3) Login as Admin.	2) Login as User: Method 1: Use Simple Form. Method 2: Use a Modular Form. Method 3: Use 3rd party services.
3. What are the necessary functionalities of Admin Dashboard? A: Admin Dashboard: 1) Perform CRUD Operation with Shipments. 2) Filter Shipments.	3) Login as Admin: Method 1: Hard Code the Admin accounts. Method 2: Have an Admin Login Page. Method 3: Have a separate Register + Login for admins.
4. What are the necessary functionalities of User Dashboard? A: User Dashboard: 1) Filter Shipments.	Admin Dashboard: 1) Perform CRUD Operation with Shipments: Method 1: Use Simple Forms and Buttons to perform CRUD. Method 2: Have separate pages for each operation Method 3: Use Modals.
5. What are the tables needed? A: Tables Required:	

- Shipments
- Users

6. What are the fields of user table?

A: Fields of Users table:

- Id (Primary Key) - int
- Name – string
- Password – string
- Role – string

7. What are the fields of the Shipment table?

A: Fields of Shipment Table:

- Id (Primary Key) - int
- Name – string
- Type – string
- Address – string

8. What are the two values of the Role Field in the User table?

A: Admin Role

User Role

9. What are the must have dependencies?

A: Dependencies:

- axios
- react-router-dom
- toastr

10. What are the validations in the application?

A: Overall validations list:

- Frontend simple JS Validations.
- Server Side validations.
- Form Validations.

2) Filter Shipments:

Method 1: Use a simple search bar.

Method 2: Have tables and sorting buttons.

Method 3: Have a separate page to pick out exact results.

User Dashboard:

1) Filter Shipments:

Method 1: Use a simple search bar.

Method 2: Have tables and sorting buttons.

Method 3: Have a separate page to pick out exact results.

Login and Registration Module:

1) Register User:

Method 1: Use Simple Form Page.

A: Since the overall scale of the application is small and data needed is rather minimal, it's wise to stick with traditional forms.

2) Login as User:

Method 1: Use Simple Form.

A: Since the overall scale of the application is small and data needed is rather minimal, it's wise to stick with traditional forms.

3) Login as Admin:

Method 1: Hard Code the Admin accounts.

A: It's best to hard code admin users to ensure security and have only one or two admin users in total.

Admin Dashboard:

1) Perform CRUD Operation with Shipments:

Method 1: Use Simple Forms and Buttons to perform CRUD.

A: Since the overall scale of the application is small and data needed is rather minimal, it's wise to stick with traditional forms.

2) Filter Shipments:

Method 1: Use a simple search bar.

A: Effective as it may, the simple input field can be directly mapped with the

Login and Registration Module:

1) Register User:

Method 2: Use a Modular Form Page.

A: Increases the complexity of the app, and also the fact that the number of needed inputs is rather minimal.

Method 3: Use Google OAuth and other 3rd party services.

A: Valid method but due to the scale of the application, this method is seemingly overkill.

2) Login as User:

Method 2: Use a Modular Form.

A: Increases the complexity of the app, and also the fact that the number of needed inputs is rather minimal.

Method 3: Use 3rd party services.

A: Valid method but due to the scale of the application, this method is seemingly overkill.

3) Login as Admin:

Method 2: Have an Admin Login Page.

A: Best not to have an admin login page distinctively when the primary user of the app is regular customers.

Method 3: Have a separate Register + Login for admins.

A: This increases the number of pages to be developed and also leaves open a vulnerable endpoint for someone to register as administrator.

shipments that are being displayed.

User Dashboard:

1) Filter Shipments:

Method 1: Use a simple search bar.

A: Effective as it may, the simple input field can be directly mapped with the shipments that are being displayed.

Admin Dashboard:

1) Perform CRUD Operation with Shipments:

Method 2: Have separate pages for each operation.

A: Since the admin needs all information in one place and also that the number of operations they can do is limited, it is best to have a single page control centre type.

Method 3: Use Modals.

A: Seeing that the number of input fields are minimal, and asking for user input using modals is seemingly overkill.

2) Filter Shipments:

Method 2: Have tables and sorting buttons.

A: This method is also viable but not selected due to the slight increase in the number of code needed for its execution.

Method 3: Have a separate page to pick out exact results.

A: Having the all the data displayed in one page and search functionality in another is not idle. Hence, this method is rejected.

User Dashboard:

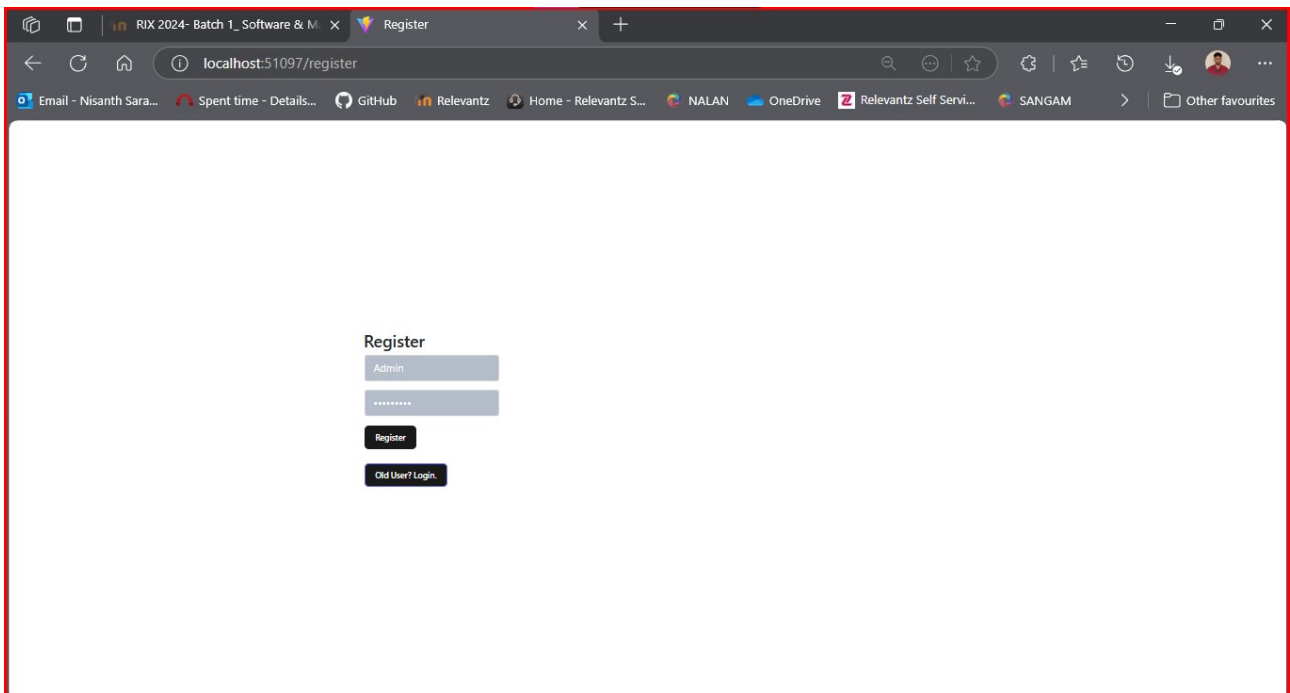
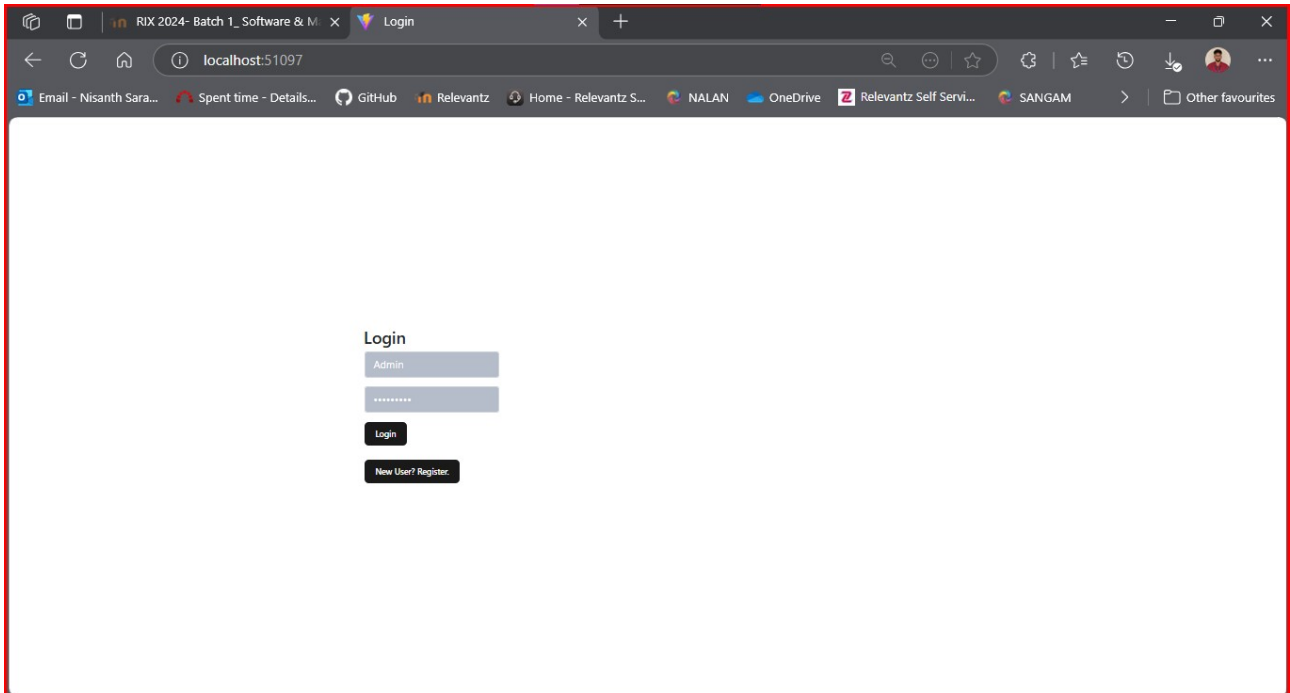
1) Filter Shipments:

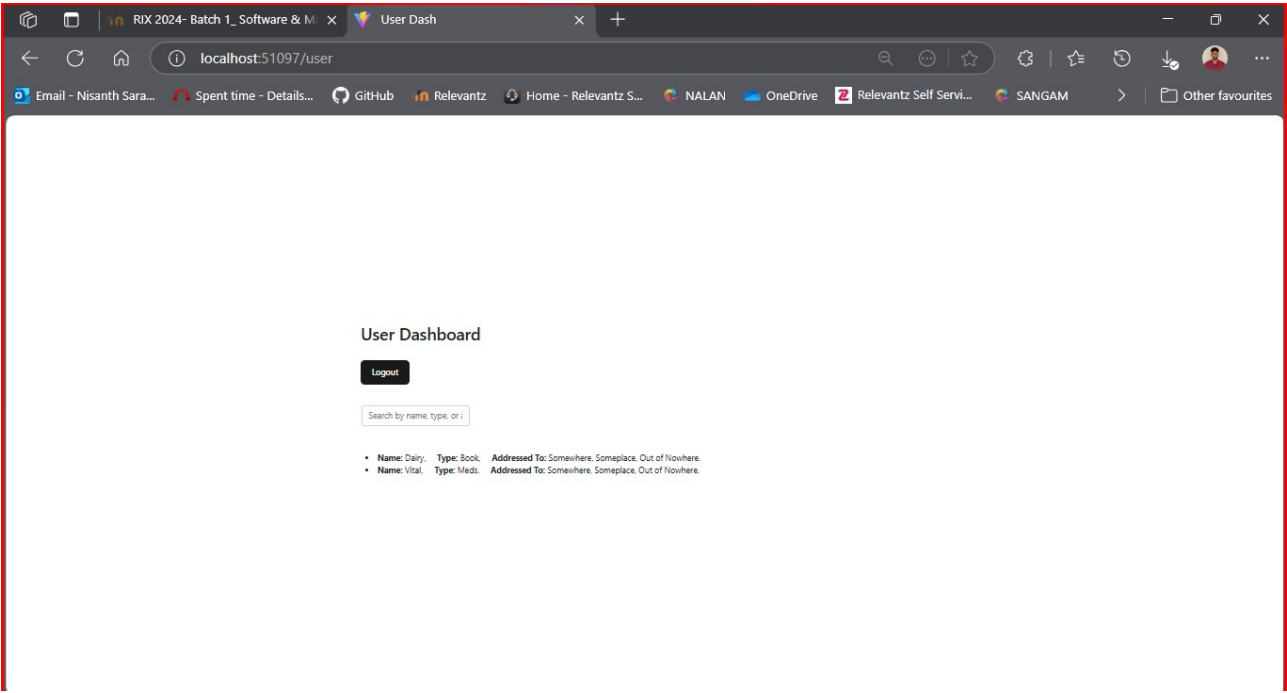
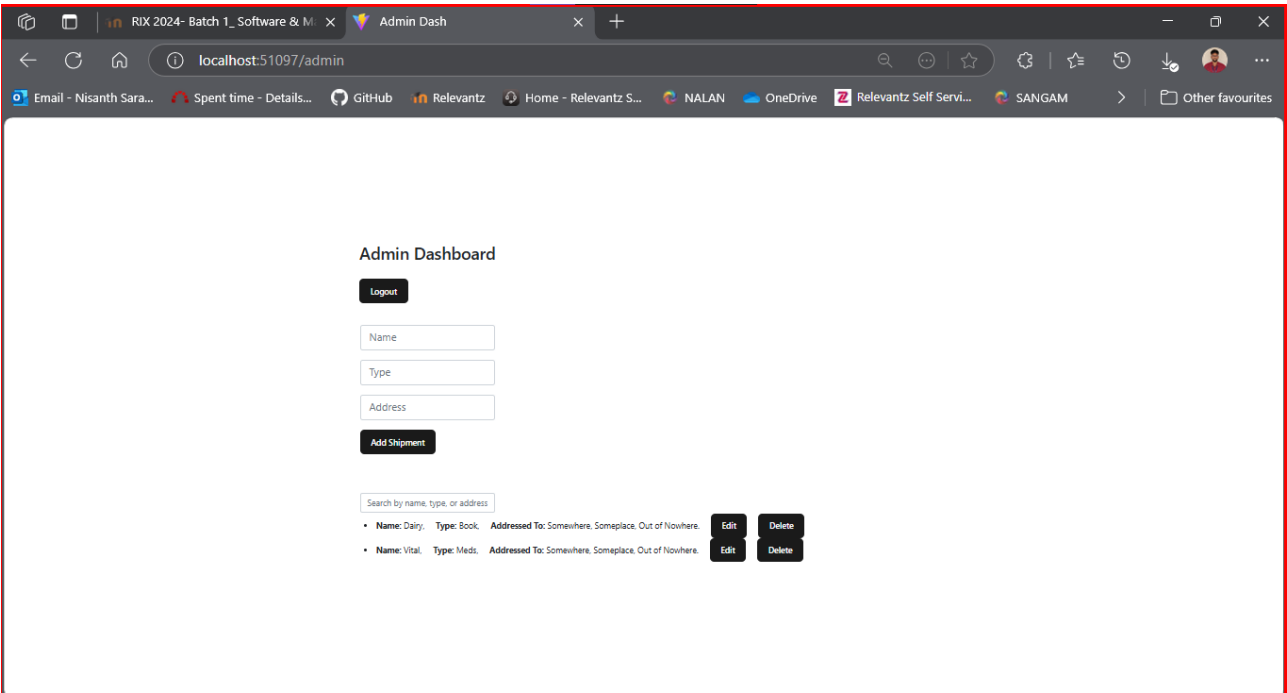
Method 2: Have tables and sorting buttons.

A: This method is also viable but not selected due to the slight increase in the number of code needed for its execution.

	Method 3: Have a separate page to pick out exact results. A: Having the all the data displayed in one page and search functionality in another is not idle. Hence, this method is rejected.
Why?	Why Not?

Output Screenshots





Testing:

Full File:



12576_Nisanth_Saravanan_NUnit&Jest_Testing.xlsx

Nunit Testing:

Project name :	Online Courier Management System
Module Name :	ShipmentController
Reference Document :	ShipmentController.cs
Created by :	Nisanth Saravanan
Date of creation :	12-08-2025
Date of review :	

Setup

```
[SetUp]
public void Setup()
{
    var options = new DbContextOptionsBuilder<AppDbContext>()
        .UseInMemoryDatabase("TestDb")
        .Options;

    _context = new AppDbContext(options);
    _controller = new ShipmentController(_context);
}
```

Tear Down

```
[TearDown]
public void TearDown()
{
    _context.Database.EnsureDeleted();
    _context.Dispose();
}
```

Usings

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using ShipmentManagementWebApi.Data;
using ShipmentManagementWebApi.Models;
```

Final Nunit Testing Output

Test Explorer			
Test Explorer			
Test run finished: 11 Tests (11 Passed, 0 Failed, 0 Warnings, 0 Errors)			
Test	Duration	Traits	Errors
ShipmentControllerTest (11)	883 ms		
ShipmentControllerTest (11)	883 ms		
ShipmentControllerTest (11)	883 ms		
Create_AdminRole_ReturnsShip...	670 ms		
Create_NonAdminRole_ReturnU...	23 ms		
Delete_AdminRole_NonExisting...	171 ms		
Delete_AdminRole_ReturnsOk	6 ms		
Delete_NonAdminRole_Returns...	< 1 ms		
GetAll_ReturnsAllShipment	9 ms		
GetById_ExistingId_ReturnsShip...	< 1 ms		
GetById_NonExistingId_Returns...	2 ms		
Update_AdminRole_NonExistin...	< 1 ms		
Update_AdminRole_ReturnsOk	2 ms		
Update_NonAdminRole_Return...	< 1 ms		

GetAll_ReturnsAllShipment()

```
[Test]
public void GetAll_ReturnsAllShipment()
{
    //Arrange
    var shipment = new Shipment
    {
        Id = 1,
        Name = "Box A",
        Type = "Electronics",
        Address = "123 Avenue"
    };
    _context.Shipments.Add(shipment);
    _context.SaveChanges();

    //Act
    var result = _controller.GetAll() as OkObjectResult;

    //Assert
    Assert.IsNotNull(result);
    Assert.IsInstanceOf<OkObjectResult>(result);
    Assert.IsNotNull(result.Value);
    Assert.IsInstanceOf<List<Shipment>>(result.Value);
    Assert.AreEqual(1, ((List<Shipment>)result.Value).Count);
}
```

GetAll_ReturnsAllShipment001	The GetAll() Method will be used to get all the existing shipments in the data base as a single List<Shipment>.	GetAll() returns an object.	Create a Valid Shipment data.	Test Database	The returned result is not null.	The returned result is not null.	Pass
			Add it to the database.	No User Role Needed			
			Call the GetAll() Method.				
GetAll_ReturnsAllShipment002	The GetAll() Method will be used to get all the existing shipments in the data base as a single List<Shipment>.	GetAll() returns an OkObjectResult	Create a Valid Shipment data.	Test Database	The returned result is an instance of OkObjectResult.	The returned result is an instance of OkObjectResult.	Pass
			Add it to the database.	No User Role Needed			
			Call the GetAll() Method.				
GetAll_ReturnsAllShipment003	The GetAll() Method will be used to get all the existing shipments in the data base as a single List<Shipment>.	GetAll() returned a not null value.	Create a Valid Shipment data.	Test Database	The returned result.Value is not null.	The returned result.Value is not null.	Pass
			Add it to the database.	No User Role Needed			
			Call the GetAll() Method.				
GetAll_ReturnsAllShipment004	The GetAll() Method will be used to get all the existing shipments in the data base as a single List<Shipment>.	GetAll() return a value that is an instance of List<Shipment>	Create a Valid Shipment data.	Test Database	The returned result.Value is an instance of List<Shipment>.	The returned result.Value is an instance of List<Shipment>.	Pass
			Add it to the database.	No User Role Needed			
			Call the GetAll() Method.				
GetAll_ReturnsAllShipment005	The GetAll() Method will be used to get all the existing shipments in the data base as a single List<Shipment>.	GetAll() - Same count as existing data.	Create a Valid Shipment data.	Test Database	The returned result.Value has the same count as the number of data within the database.	The returned result.Value has the same count as the number of data within the database.	Pass
			Add it to the database.	No User Role Needed			
			Call the GetAll() Method.				

GetById_ExistingId_ReturnsShipment()

```
[Test]
public void GetById_ExistingId_ReturnsShipment()
{
    //Arrange
    var shipment = new Shipment
    {
        Id = 1,
        Name = "Box A",
        Type = "Electronics",
        Address = "123 Avenue"
    };
    _context.Shipments.Add(shipment);
    _context.SaveChanges();

    //Act
    var result = _controller.Get(1) as OkObjectResult;

    //Assert
    Assert.IsNotNull(result);
    Assert.IsInstanceOf<OkObjectResult>(result);
    Assert.IsNotNull(result.Value);
    Assert.IsInstanceOf<Shipment>(result.Value);
    Assert.AreEqual(shipment, result.Value);
}
```

GetById_ExistingId_ReturnsShipment1001	The GetById(int id) Method will be used to get a single shipment data in the data base as an instance of Shipment.	GetById() returns an object.	Create a Valid Shipment data.	Test Database	The returned result is not null.	The returned result is not null.	Pass
			Add it to the database.	No User Role Needed			
			Call the GetById(id) method.	Valid id			
GetById_ExistingId_ReturnsShipment1002	The GetById(int id) Method will be used to get a single shipment data in the data base as an instance of Shipment.	GetById() returns an OkObjectResult.	Create a Valid Shipment data.	Test Database	The returned result is an instance of OkObjectResult.	The returned result is an instance of OkObjectResult.	Pass
			Add it to the database.	No User Role Needed			
			Call the GetById(id) method.	Valid id			
GetById_ExistingId_ReturnsShipment1003	The GetById(int id) Method will be used to get a single shipment data in the data base as an instance of Shipment.	GetById() returned a not null value.	Create a Valid Shipment data.	Test Database	The returned result.Value is not null.	The returned result.Value is not null.	Pass
			Add it to the database.	No User Role Needed			
			Call the GetById(id) method.	Valid id			
GetById_ExistingId_ReturnsShipment1004	The GetById(int id) Method will be used to get a single shipment data in the data base as an instance of Shipment.	GetById() returned value is an instance of Shipment.			The returned result.Value is an instance of Shipment.	The returned result.Value is an instance of Shipment.	Pass
GetById_ExistingId_ReturnsShipment1005	The GetById(int id) Method will be used to get a single shipment data in the data base as an instance of Shipment.	GetById() returns the correct shipment data.	Create a Valid Shipment data.	Test Database	The returned result.Value is equal to the shipment value that was inserted.	The returned result.Value is equal to the shipment value that was inserted.	Pass
			Add it to the database.	No User Role Needed			
			Call the GetById(id) method.	Valid id			

GetById_NonExistingId_ReturnsNotFound()

```
[Test]
public void GetById_NonExistingId_ReturnsNotFound()
{
    //Act
    var result = _controller.Get(1) as NotFoundObjectResult;

    //Assert
    Assert.IsNotNull(result);
    Assert.IsInstanceOf<NotFoundObjectResult>(result);
    Assert.IsNotNull(result.Value);
    Assert.IsInstanceOf<String>(result.Value);
    Assert.AreEqual("Shipment Not Found!", result.Value);
}
```

GetById_NonExistingId_ReturnsNotFound001	The GetById(int id) method is called with a non-existing id.	GetById() returns an object.	Call the GetById(id) method.	Test Database	The returned result is not null.	The returned result is not null.	Pass
				No User Role Needed			
				Invalid id			
GetById_NonExistingId_ReturnsNotFound002	The GetById(int id) method is called with a non-existing id.	GetById() returns an NotFoundObjectResult.	Call the GetById(id) method.	Test Database	The returned result is an instance of NotFoundObjectResult.	The returned result is an instance of NotFoundObjectResult.	Pass
				No User Role Needed			
				Invalid id			
GetById_NonExistingId_ReturnsNotFound003	The GetById(int id) method is called with a non-existing id.	GetById() returned a not null value.	Call the GetById(id) method.	Test Database	The returned result.Value is not null.	The returned result.Value is not null.	Pass
				No User Role Needed			
				Invalid id			
GetById_NonExistingId_ReturnsNotFound004	The GetById(int id) method is called with a non-existing id.	GetById() returned value is an instance of String.	Call the GetById(id) method.	Test Database	The returned result.Value is an instance of <String>.	The returned result.Value is an instance of <String>.	Pass
				No User Role Needed			
				Invalid id			
GetById_NonExistingId_ReturnsNotFound005	The GetById(int id) method is called with a non-existing id.	GetById() returns the correct response message.	Call the GetById(id) method.	Test Database	The returned result.Value is equal to the preset response message.	The returned result.Value is equal to the preset response message.	Pass
				No User Role Needed			
				Invalid id			

Create_AdminRole_ReturnsShipment()

```
[Test]
public void Create_AdminRole_ReturnsShipment()
{
    //Arrange
    var shipment = new Shipment
    {
        Id = 1,
        Name = "Box A",
        Type = "Electronics",
        Address = "123 Avenue"
    };

    //Act
    var result = _controller.Create(shipment, "Admin") as OkObjectResult;

    //Assert
    Assert.IsNotNull(result);
    Assert.IsInstanceOf<OkObjectResult>(result);
    Assert.IsNotNull(result.Value);
    Assert.IsInstanceOf<Shipment>(result.Value);
    Assert.AreEqual(shipment, result.Value);
}
```

Create_AdminRole_ReturnsShipment1001	The Create() method is called with valid shipment data and Admin role to create a new shipment record in the database.	Create() returns an object.	Create a Valid Shipment data.	Test Database	The returned result is not null.	The returned result is not null.	Pass
			Call Create() Method.	Logged in as Admin.			
				Valid Shipment Data.			
Create_AdminRole_ReturnsShipment1002	The Create() method is called with valid shipment data and Admin role to create a new shipment record in the database.	Create() returns an instance of OkObjectResult.	Create a Valid Shipment data.	Test Database	The returned result is an instance of OkObjectResult.	The returned result is an instance of OkObjectResult.	Pass
			Call Create() Method.	Logged in as Admin.			
				Valid Shipment Data.			
Create_AdminRole_ReturnsShipment1003	The Create() method is called with valid shipment data and Admin role to create a new shipment record in the database.	Create() returns a not null value.	Create a Valid Shipment data.	Test Database	The returned result.Value is not null.	The returned result.Value is not null.	Pass
			Call Create() Method.	Logged in as Admin.			
				Valid Shipment Data.			
Create_AdminRole_ReturnsShipment1004	The Create() method is called with valid shipment data and Admin role to create a new shipment record in the database.	Create() returns an instance of Shipment.	Create a Valid Shipment data.	Test Database	The returned result.Value is an instance of Shipment.	The returned result.Value is an instance of Shipment.	Pass
			Call Create() Method.	Logged in as Admin.			
				Valid Shipment Data.			
Create_AdminRole_ReturnsShipment1005	The Create() method is called with valid shipment data and Admin role to create a new shipment record in the database.	Create() returns the correct shipment data.	Create a Valid Shipment data.	Test Database	The returned result.Value is equal to the shipment value that was inserted.	The returned result.Value is equal to the shipment value that was inserted.	Pass
			Call Create() Method.	Logged in as Admin.			
				Valid Shipment Data.			

Create_NonAdminRole_ReturnUnauthorized()

```
[Test]
public void Create_NonAdminRole_ReturnUnauthorized()
{
    //Arrange
    var shipment = new Shipment
    {
        Id = 1,
        Name = "Box A",
        Type = "Electronics",
        Address = "123 Avenue"
    };

    //Act
    var result = _controller.Create(shipment, "User") as UnauthorizedObjectResult;

    //Assert
    Assert.IsNotNull(result);
    Assert.IsInstanceOf<UnauthorizedObjectResult>(result);
    Assert.IsNotNull(result.Value);
    Assert.IsInstanceOf<String>(result.Value);
    Assert.AreEqual("Only admin can create shipments", result.Value);
}
```

Create_NonAdminRole_ReturnUnauthorized001	The Create() method is called with valid shipment data and User role to create a new shipment record in the database.	Create() returns an object.	Create a Valid Shipment data.	Test Database	The returned result is not null.	The returned result is not null.	Pass
			Call Create() Method.	Logged in as User.			
				Valid Shipment Data.			
Create_NonAdminRole_ReturnUnauthorized002	The Create() method is called with valid shipment data and User role to create a new shipment record in the database.	Create() returns an UnauthorizedObjectResult.	Create a Valid Shipment data.	Test Database	The returned result is an instance of UnauthorizedObjectResult.	The returned result is an instance of UnauthorizedObjectResult.	Pass
			Call Create() Method.	Logged in as User.			
				Valid Shipment Data.			
Create_NonAdminRole_ReturnUnauthorized003	The Create() method is called with valid shipment data and User role to create a new shipment record in the database.	Create() returned a not null value.	Create a Valid Shipment data.	Test Database	The returned result.Value is not null.	The returned result.Value is not null.	Pass
			Call Create() Method.	Logged in as User.			
				Valid Shipment Data.			
Create_NonAdminRole_ReturnUnauthorized004	The Create() method is called with valid shipment data and User role to create a new shipment record in the database.	Create() returned value is an instance of String.	Create a Valid Shipment data.	Test Database	The returned result.Value is an instance of <String>.	The returned result.Value is an instance of <String>.	Pass
			Call Create() Method.	Logged in as User.			
				Valid Shipment Data.			
Create_NonAdminRole_ReturnUnauthorized005	The Create() method is called with valid shipment data and User role to create a new shipment record in the database.	Create() returns the correct response message.	Create a Valid Shipment data.	Test Database	The returned result.Value is equal to the preset response message.	The returned result.Value is equal to the preset response message.	Pass
			Call Create() Method.	Logged in as User.			
				Valid Shipment Data.			

Update_AdminRole_ReturnsOk()

```
[Test]
public void Update_AdminRole_ReturnsOk()
{
    //Arrange
    var shipment = new Shipment
    {
        Id = 1,
        Name = "Box A",
        Type = "Electronics",
        Address = "123 Avenue"
    };
    _context.Shipments.Add(shipment);
    _context.SaveChanges();

    var updatedShipment = new Shipment
    {
        Id = 1,
        Name = "Box A",
        Type = "Misc",
        Address = "123 Avenue"
    };

    //Act
    var result = _controller.Update(1, updatedShipment, "Admin") as OkObjectResult;

    //Assert
    Assert.IsNotNull(result);
    Assert.IsInstanceOf<OkObjectResult>(result);
    Assert.IsNotNull(result.Value);
    Assert.IsInstanceOf<Shipment>(result.Value);
    var resultingShipment = result.Value as Shipment;
    Assert.AreEqual(updatedShipment.Type, resultingShipment.Type);
}
```

Update_AdminRole_ReturnsOk001	The Update() method is called with a valid id, valid shipment data and with Admin role to update an existing shipment.	Update() returns an object.	Create a Valid Shipment data.	Test Database	The returned result is not null.	The returned result is not null.	Pass
			Add it to the database.	Logged in as Admin.			
			Create a Valid Updated Shipment data under the same ID.	Valid Shipment Data.			
			Call Update() Method.	Valid id.			
Update_AdminRole_ReturnsOk002	The Update() method is called with a valid id, valid shipment data and with Admin role to update an existing shipment.	Update() returns an instance of OkObjectResult.	Create a Valid Shipment data.	Test Database	The returned result is an instance of OkObjectResult.	The returned result is an instance of OkObjectResult.	Pass
			Add it to the database.	Logged in as Admin.			
			Create a Valid Updated Shipment data under the same ID.	Valid Shipment Data.			
			Call Update() Method.	Valid id.			
Update_AdminRole_ReturnsOk003	The Update() method is called with a valid id, valid shipment data and with Admin role to update an existing shipment.	Update() returns a not null value.	Add it to the database.	Logged in as Admin.			
			Create a Valid Updated Shipment data under the same ID.	Valid Shipment Data.			
			Call Update() Method.	Valid id.			
Update_AdminRole_ReturnsOk004	The Update() method is called with a valid id, valid shipment data and with Admin role to update an existing shipment.	Update() returns an instance of Shipment.	Create a Valid Shipment data.	Test Database	The returned result.Value is an instance of Shipment.	The returned result.Value is an instance of Shipment.	Pass
			Add it to the database.	Logged in as Admin.			
			Create a Valid Updated Shipment data under the same ID.	Valid Shipment Data.			
			Call Update() Method.	Valid id.			
Update_AdminRole_ReturnsOk005	The Update() method is called with a valid id, valid shipment data and with Admin role to update an existing shipment.	Update() returns the correct updated shipment data.	Create a Valid Shipment data.	Test Database	The returned result.Value is equal to the shipment value that was updated.	The returned result.Value is equal to the shipment value that was updated.	Pass
			Add it to the database.	Logged in as Admin.			
			Create a Valid Updated Shipment data under the same ID.	Valid Shipment Data.			
			Call Update() Method.	Valid id.			

```

        Update_NonAdminRole_ReturnsUnauthorized()[Test]
public void Update_NonAdminRole_ReturnsUnauthorized()
{
    //Arrange
    var shipment = new Shipment
    {
        Id = 1,
        Name = "Box A",
        Type = "Electronics",
        Address = "123 Avenue"
    };
    _context.Shipments.Add(shipment);
    _context.SaveChanges();

    var updatedShipment = new Shipment
    {
        Id = 1,
        Name = "Box A",
        Type = "Misc",
        Address = "123 Avenue"
    };

    //Act
    var result = _controller.Update(1, updatedShipment, "User") as
UnauthorizedObjectResult;

    //Assert
    Assert.IsNotNull(result);
    Assert.IsInstanceOf<UnauthorizedObjectResult>(result);
    Assert.IsNotNull(result.Value);
    Assert.IsInstanceOf<String>(result.Value);
    Assert.AreEqual("Only admin can update shipments", result.Value);
}

```

Update_NonAdminRole_ReturnsUnauthorized001	The Update() method is called with a valid id, valid shipment data and with User role to update an existing shipment.	Update() returns an object.	Add it to the database.	Logged in as User.			
			Create a Valid Updated Shipment data under the same ID.	Valid Shipment Data.			
			Call Update() Method.	Valid id.			
Update_NonAdminRole_ReturnsUnauthorized002	The Update() method is called with a valid id, valid shipment data and with User role to update an existing shipment.	Update() returns an UnauthorizedObjectResult.	Create a Valid Shipment data.	Test Database	The returned result is an instance of UnauthorizedObjectResult.	The returned result is an instance of UnauthorizedObjectResult.	Pass
			Add it to the database.	Logged in as User.			
			Create a Valid Updated Shipment data under the same ID.	Valid Shipment Data.			
			Call Update() Method.	Valid id.			
Update_NonAdminRole_ReturnsUnauthorized003	The Update() method is called with a valid id, valid shipment data and with User role to update an existing shipment.	Update() returned a not null value.	Create a Valid Shipment data.	Test Database	The returned result.Value is not null.	The returned result.Value is not null.	Pass
			Add it to the database.	Logged in as User.			
			Create a Valid Updated Shipment data under the same ID.	Valid Shipment Data.			
			Call Update() Method.	Valid id.			
Update_NonAdminRole_ReturnsUnauthorized004	The Update() method is called with a valid id, valid shipment data and with User role to update an existing shipment.	Update() returned value is an instance of String.	Create a Valid Shipment data.	Test Database	The returned result.Value is an instance of <String>.	The returned result.Value is an instance of <String>.	Pass
			Add it to the database.	Logged in as User.			
			Create a Valid Updated Shipment data under the same ID.	Valid Shipment Data.			
			Call Update() Method.	Valid id.			
Update_NonAdminRole_ReturnsUnauthorized005	The Update() method is called with a valid id, valid shipment data and with User role to update an existing shipment.	Update() returns the correct response message.	Create a Valid Shipment data.	Test Database	The returned result.Value is equal to the preset response message.	The returned result.Value is equal to the preset response message.	Pass
			Add it to the database.	Logged in as User.			
			Create a Valid Updated Shipment data under the same ID.	Valid Shipment Data.			
			Call Update() Method.	Valid id.			

Update_AdminRole_NonExistingId_ReturnsNotFound()

```
[Test]
public void Update_AdminRole_NonExistingId_ReturnsNotFound()
{
    //Arrange
    var updatedShipment = new Shipment
    {
        Id = 1,
        Name = "Box A",
        Type = "Misc",
        Address = "123 Avenue"
    };

    //Act
    var result = _controller.Update(1, updatedShipment, "Admin") as
    NotFoundObjectResult;

    //Assert
    Assert.IsNotNull(result);
    Assert.IsInstanceOf<NotFoundObjectResult>(result);
    Assert.IsNotNull(result.Value);
    Assert.IsInstanceOf<String>(result.Value);
    Assert.AreEqual("Shipment Not Found!", result.Value);
}
```

Update_AdminRole_NonExistingId_ReturnsNotFound01	The Update() method is called with a invalid Id, valid shipment data and with Admin role to update an non-existing shipment.	Update() returns an object.	Create a Valid Shipment data.	Test Database	The returned result is not null.	The returned result is not null.	Pass
			Add it to the database.	Logged in as Admin.			
			Create a Valid Updated Shipment data under the same ID.	Valid Shipment Data.			
			Call Update() Method.	Invalid ID.			
Update_AdminRole_NonExistingId_ReturnsNotFound02	The Update() method is called with a invalid Id, valid shipment data and with Admin role to update an non-existing shipment.	Update() returns an NotFoundObjectResult.	Create a Valid Shipment data.	Test Database	The returned result is an instance of NotFoundObjectResult.	The returned result is an instance of NotFoundObjectResult.	Pass
			Add it to the database.	Logged in as Admin.			
			Create a Valid Updated Shipment data under the same ID.	Valid Shipment Data.			
			Call Update() Method.	Invalid ID.			
Update_AdminRole_NonExistingId_ReturnsNotFound03	The Update() method is called with a invalid Id, valid shipment data and with Admin role to update an non-existing shipment.	Update() returned a not null value.	Create a Valid Shipment data.	Test Database	The returned result.Value is not null.	The returned result.Value is not null.	Pass
			Add it to the database.	Logged in as Admin.			
			Create a Valid Updated Shipment data under the same ID.	Valid Shipment Data.			
			Call Update() Method.	Invalid ID.			
Update_AdminRole_NonExistingId_ReturnsNotFound04	The Update() method is called with a invalid Id, valid shipment data and with Admin role to update an non-existing shipment.	Update() returned value is an instance of String.	Create a Valid Shipment data.	Test Database	The returned result.Value is an instance of <String>.	The returned result.Value is an instance of <String>.	Pass
			Add it to the database.	Logged in as Admin.			
			Create a Valid Updated Shipment data under the same ID.	Valid Shipment Data.			
			Call Update() Method.	Invalid ID.			
Update_AdminRole_NonExistingId_ReturnsNotFound05	The Update() method is called with a invalid Id, valid shipment data and with Admin role to update an non-existing shipment.	Update() returns the correct response message.	Create a Valid Shipment data.	Test Database	The returned result.Value is equal to the preset response message.	The returned result.Value is equal to the preset response message.	Pass
			Add it to the database.	Logged in as Admin.			
			Create a Valid Updated Shipment data under the same ID.	Valid Shipment Data.			
			Call Update() Method.	Invalid ID.			

Delete_AdminRole_ReturnsOk()

```
[Test]
public void Delete_AdminRole_ReturnsOk()
{
    //Arrange
    var shipment = new Shipment
    {
        Id = 1,
        Name = "Box A",
        Type = "Electronics",
        Address = "123 Avenue"
    };
    _context.Shipments.Add(shipment);
    _context.SaveChanges();

    //Act
    var result = _controller.Delete(1, "Admin") as OkObjectResult;

    //Assert
    Assert.IsNotNull(result);
    Assert.IsInstanceOf<OkObjectResult>(result);
    Assert.IsNotNull(result.Value);
    Assert.IsInstanceOf<String>(result.Value);
    Assert.AreEqual("Deleted", result.Value);
}
```

Delete_AdminRole_ReturnsOk001	The Delete() method is called with valid id and with Admin role to delete an existing shipment data.	Delete() returns an object.	Create a Valid Shipment data.	Test Database	The returned result is not null.	The returned result is not null.	Pass
			Add it to the database.	Logged in as Admin.			
			Call Delete() Method.	Valid Shipment Data.			
				Valid ID.			
Delete_AdminRole_ReturnsOk002	The Delete() method is called with valid id and with Admin role to delete an existing shipment data.	Delete() returns an OkObjectResult.	Create a Valid Shipment data.	Test Database	The returned result is an instance of OkObjectResult.	The returned result is an instance of OkObjectResult.	Pass
			Add it to the database.	Logged in as Admin.			
			Call Delete() Method.	Valid Shipment Data.			
				Valid ID.			
Delete_AdminRole_ReturnsOk003	The Delete() method is called with valid id and with Admin role to delete an existing shipment data.	Delete() returned a not null value.	Create a Valid Shipment data.	Test Database	The returned result.Value is not null.	The returned result.Value is not null.	Pass
			Add it to the database.	Logged in as Admin.			
			Call Delete() Method.	Valid Shipment Data.			
				Valid ID.			
Delete_AdminRole_ReturnsOk004	The Delete() method is called with valid id and with Admin role to delete an existing shipment data.	Delete() returned value is an instance of String.	Create a Valid Shipment data.	Test Database	The returned result.Value is an instance of <String>.	The returned result.Value is an instance of <String>.	Pass
			Add it to the database.	Logged in as Admin.			
			Call Delete() Method.	Valid Shipment Data.			
				Valid ID.			
Delete_AdminRole_ReturnsOk005	The Delete() method is called with valid id and with Admin role to delete an existing shipment data.	Delete() returns the correct response message.	Create a Valid Shipment data.	Test Database	The returned result.Value is equal to the preset response message.	The returned result.Value is equal to the preset response message.	Pass
			Add it to the database.	Logged in as Admin.			
			Call Delete() Method.	Valid Shipment Data.			
				Valid ID.			

Delete_NonAdminRole_ReturnsUnauthorized()

```
[Test]
public void Delete_NonAdminRole_ReturnsUnauthorized()
{
    //Arrange
    var shipment = new Shipment
    {
        Id = 1,
        Name = "Box A",
        Type = "Electronics",
        Address = "123 Avenue"
    };
    _context.Shipments.Add(shipment);
    _context.SaveChanges();

    //Act
    var result = _controller.Delete(1, "User") as UnauthorizedObjectResult;

    //Assert
    Assert.IsNotNull(result);
    Assert.IsInstanceOf<UnauthorizedObjectResult>(result);
    Assert.IsNotNull(result.Value);
    Assert.IsInstanceOf<String>(result.Value);
    Assert.AreEqual("Only admin can delete shipments", result.Value);
}
```

Delete_NonAdminRole_ReturnsUnauthorized001	The Delete() method is called with valid id and with User role to delete an existing shipment data.	Delete() returns an object.	Add it to the database.	Logged in as User.			
			Call Delete() Method.	Valid Shipment Data.			
				Valid ID.			
Delete_NonAdminRole_ReturnsUnauthorized002	The Delete() method is called with valid id and with User role to delete an existing shipment data.	Delete() returns an UnauthorizedObjectResult.	Create a Valid Shipment data.	Test Database	The returned result is an instance of UnauthorizedObjectResult.	The returned result is an instance of UnauthorizedObjectResult.	Pass
			Add it to the database.	Logged in as User.			
			Call Delete() Method.	Valid Shipment Data.			
				Valid ID.			
Delete_NonAdminRole_ReturnsUnauthorized003	The Delete() method is called with valid id and with User role to delete an existing shipment data.	Delete() returned a not null value.	Create a Valid Shipment data.	Test Database	The returned result.Value is not null.	The returned result.Value is not null.	Pass
			Add it to the database.	Logged in as User.			
			Call Delete() Method.	Valid Shipment Data.			
				Valid ID.			
Delete_NonAdminRole_ReturnsUnauthorized004	The Delete() method is called with valid id and with User role to delete an existing shipment data.	Delete() returned value is an instance of String.	Create a Valid Shipment data.	Test Database	The returned result.Value is an instance of <String>.	The returned result.Value is an instance of <String>.	Pass
			Add it to the database.	Logged in as User.			
			Call Delete() Method.	Valid Shipment Data.			
				Valid ID.			
Delete_NonAdminRole_ReturnsUnauthorized005	The Delete() method is called with valid id and with User role to delete an existing shipment data.	Delete() returns the correct response message.	Create a Valid Shipment data.	Test Database	The returned result.Value is equal to the preset response message.	The returned result.Value is equal to the preset response message.	Pass
			Add it to the database.	Logged in as User.			
			Call Delete() Method.	Valid Shipment Data.			
				Valid ID.			

Delete_AdminRole_NonExistingId_ReturnsNotFound()

```
[Test]
public void Delete_AdminRole_NonExistingId_ReturnsNotFound()
{
    //Act
    var result = _controller.Delete(1, "Admin") as NotFoundObjectResult;

    //Assert
    Assert.IsNotNull(result);
    Assert.IsInstanceOf<NotFoundObjectResult>(result);
    Assert.IsNotNull(result.Value);
    Assert.IsInstanceOf<String>(result.Value);
    Assert.AreEqual("Shipment Not Found!", result.Value);
}
```

Delete_AdminRole_NonExistingId_ReturnsNotFound01	The Delete() method is called with invalid id and with Admin role to delete a non-existing shipment data.	Delete() returns an object.	Add it to the database.	Logged in as Admin.			
			Call Delete() Method.	Valid Shipment Data.			
				Invalid ID.			
Delete_AdminRole_NonExistingId_ReturnsNotFound02	The Delete() method is called with invalid id and with Admin role to delete a non-existing shipment data.	Delete() returns an NotFoundObjectResult .	Create a Valid Shipment data.	Test Database	The returned result is an instance of NotFoundObjectResult .	The returned result is an instance of NotFoundObjectResult .	Pass
			Add it to the database.	Logged in as Admin.			
			Call Delete() Method.	Valid Shipment Data.			
				Invalid ID.			
Delete_AdminRole_NonExistingId_ReturnsNotFound03	The Delete() method is called with invalid id and with Admin role to delete a non-existing shipment data.	Delete() returned a not null value.	Create a Valid Shipment data.	Test Database	The returned result.Value is not null.	The returned result.Value is not null.	Pass
			Add it to the database.	Logged in as Admin.			
			Call Delete() Method.	Valid Shipment Data.			
				Invalid ID.			
Delete_AdminRole_NonExistingId_ReturnsNotFound04	The Delete() method is called with invalid id and with Admin role to delete a non-existing shipment data.	Delete() returned value is an instance of String.	Create a Valid Shipment data.	Test Database	The returned result.Value is an instance of <String>.	The returned result.Value is an instance of <String>.	Pass
			Add it to the database.	Logged in as Admin.			
			Call Delete() Method.	Valid Shipment Data.			
				Invalid ID.			
Delete_AdminRole_NonExistingId_ReturnsNotFound05	The Delete() method is called with invalid id and with Admin role to delete a non-existing shipment data.	Delete() returns the correct response message.	Create a Valid Shipment data.	Test Database	The returned result.Value is equal to the preset response message.	The returned result.Value is equal to the preset response message.	Pass
			Add it to the database.	Logged in as Admin.			
			Call Delete() Method.	Valid Shipment Data.			
				Invalid ID.			

Full Nunit Testing Code

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using ShipmentManagementWebApi.Data;
using ShipmentManagementWebApi.Models;

namespace ShipmentControllerTest
{
    [TestFixture]
    public class ShipmentControllerTest
    {
        private AppDbContext _context;
        private ShipmentController _controller;

        [SetUp]
        public void Setup()
        {
            var options = new DbContextOptionsBuilder<AppDbContext>()
                .UseInMemoryDatabase("TestDb")
                .Options;

            _context = new AppDbContext(options);
            _controller = new ShipmentController(_context);
        }

        [TearDown]
        public void TearDown()
        {
            _context.Database.EnsureDeleted();
            _context.Dispose();
        }

        [Test]
        public void GetAll_ReturnsAllShipment()
        {
            //Arrange
            var shipment = new Shipment
            {
                Id = 1,
                Name = "Box A",
                Type = "Electronics",
                Address = "123 Avenue"
            };
            _context.Shipments.Add(shipment);
            _context.SaveChanges();

            //Act
            var result = _controller.GetAll() as OkObjectResult;

            //Assert
            Assert.IsNotNull(result);
            Assert.IsInstanceOf<OkObjectResult>(result);
            Assert.IsNotNull(result.Value);
            Assert.IsInstanceOf<List<Shipment>>(result.Value);
            Assert.AreEqual(1, ((List<Shipment>)result.Value).Count);
        }

        [Test]
        public void GetById_ExistingId_ReturnsShipment()
        {
            //Arrange
            var shipment = new Shipment
            {

```

```

        Id = 1,
        Name = "Box A",
        Type = "Electronics",
        Address = "123 Avenue"
    };
    _context.Shipments.Add(shipment);
    _context.SaveChanges();

    //Act
    var result = _controller.Get(1) as OkObjectResult;

    //Assert
    Assert.IsNotNull(result);
    Assert.IsInstanceOf<OkObjectResult>(result);
    Assert.IsNotNull(result.Value);
    Assert.IsInstanceOf<Shipment>(result.Value);
    Assert.AreEqual(shipment, result.Value);
}

[Test]
public void GetById_NonExistingId_ReturnsNotFound()
{
    //Act
    var result = _controller.Get(1) as NotFoundObjectResult;

    //Assert
    Assert.IsNotNull(result);
    Assert.IsInstanceOf<NotFoundObjectResult>(result);
    Assert.IsNotNull(result.Value);
    Assert.IsInstanceOf<String>(result.Value);
    Assert.AreEqual("Shipment Not Found!", result.Value);
}

[Test]
public void Create_AdminRole_ReturnsShipment()
{
    //Arrange
    var shipment = new Shipment
    {
        Id = 1,
        Name = "Box A",
        Type = "Electronics",
        Address = "123 Avenue"
    };

    //Act
    var result = _controller.Create(shipment, "Admin") as OkObjectResult;

    //Assert
    Assert.IsNotNull(result);
    Assert.IsInstanceOf<OkObjectResult>(result);
    Assert.IsNotNull(result.Value);
    Assert.IsInstanceOf<Shipment>(result.Value);
    Assert.AreEqual(shipment, result.Value);
}

[Test]
public void Create_NonAdminRole_ReturnUnauthorized()
{
    //Arrange
    var shipment = new Shipment
    {
        Id = 1,
        Name = "Box A",
        Type = "Electronics",

```



```

        Address = "123 Avenue"
    };

    //Act
    var result = _controller.Create(shipment, "User") as
UnauthorizedObjectResult;

    //Assert
    Assert.IsNotNull(result);
    Assert.IsInstanceOf<UnauthorizedObjectResult>(result);
    Assert.IsNotNull(result.Value);
    Assert.IsInstanceOf<String>(result.Value);
    Assert.AreEqual("Only admin can create shipments", result.Value);
}

[Test]
public void Update_AdminRole_ReturnsOk()
{
    //Arrange
    var shipment = new Shipment
    {
        Id = 1,
        Name = "Box A",
        Type = "Electronics",
        Address = "123 Avenue"
    };
    _context.Shipments.Add(shipment);
    _context.SaveChanges();

    var updatedShipment = new Shipment
    {
        Id = 1,
        Name = "Box A",
        Type = "Misc",
        Address = "123 Avenue"
    };

    //Act
    var result = _controller.Update(1, updatedShipment, "Admin") as
OkObjectResult;

    //Assert
    Assert.IsNotNull(result);
    Assert.IsInstanceOf<OkObjectResult>(result);
    Assert.IsNotNull(result.Value);
    Assert.IsInstanceOf<Shipment>(result.Value);
    var resultingShipment = result.Value as Shipment;
    Assert.AreEqual(updatedShipment.Type, resultingShipment.Type);
}

[Test]
public void Update_NonAdminRole_ReturnsUnauthorized()
{
    //Arrange
    var shipment = new Shipment
    {
        Id = 1,
        Name = "Box A",
        Type = "Electronics",
        Address = "123 Avenue"
    };
    _context.Shipments.Add(shipment);
    _context.SaveChanges();

    var updatedShipment = new Shipment

```

```

        {
            Id = 1,
            Name = "Box A",
            Type = "Misc",
            Address = "123 Avenue"
        };

        //Act
        var result = _controller.Update(1, updatedShipment, "User") as
UnauthorizedObjectResult;

        //Assert
        Assert.IsNotNull(result);
        Assert.IsInstanceOf<UnauthorizedObjectResult>(result);
        Assert.IsNotNull(result.Value);
        Assert.IsInstanceOf<String>(result.Value);
        Assert.AreEqual("Only admin can update shipments", result.Value);
    }

    [Test]
    public void Update_AdminRole_NonExistingId_ReturnsNotFound()
    {
        //Arrange
        var updatedShipment = new Shipment
        {
            Id = 1,
            Name = "Box A",
            Type = "Misc",
            Address = "123 Avenue"
        };

        //Act
        var result = _controller.Update(1, updatedShipment, "Admin") as
NotFoundObjectResult;

        //Assert
        Assert.IsNotNull(result);
        Assert.IsInstanceOf<NotFoundObjectResult>(result);
        Assert.IsNotNull(result.Value);
        Assert.IsInstanceOf<String>(result.Value);
        Assert.AreEqual("Shipment Not Found!", result.Value);
    }

    [Test]
    public void Delete_AdminRole_ReturnsOk()
    {
        //Arrange
        var shipment = new Shipment
        {
            Id = 1,
            Name = "Box A",
            Type = "Electronics",
            Address = "123 Avenue"
        };
        _context.Shipments.Add(shipment);
        _context.SaveChanges();

        //Act
        var result = _controller.Delete(1, "Admin") as OkObjectResult;

        //Assert
        Assert.IsNotNull(result);
        Assert.IsInstanceOf<OkObjectResult>(result);
        Assert.IsNotNull(result.Value);
        Assert.IsInstanceOf<String>(result.Value);
    }

```

```

        Assert.AreEqual("Deleted", result.Value);
    }

[Test]
public void Delete_NonAdminRole_ReturnsUnauthorized()
{
    //Arrange
    var shipment = new Shipment
    {
        Id = 1,
        Name = "Box A",
        Type = "Electronics",
        Address = "123 Avenue"
    };
    _context.Shipments.Add(shipment);
    _context.SaveChanges();

    //Act
    var result = _controller.Delete(1, "User") as UnauthorizedObjectResult;

    //Assert
    Assert.IsNotNull(result);
    Assert.IsInstanceOf<UnauthorizedObjectResult>(result);
    Assert.IsNotNull(result.Value);
    Assert.IsInstanceOf<String>(result.Value);
    Assert.AreEqual("Only admin can delete shipments", result.Value);
}

[Test]
public void Delete_AdminRole_NonExistingId_ReturnsNotFound()
{
    //Act
    var result = _controller.Delete(1, "Admin") as NotFoundObjectResult;

    //Assert
    Assert.IsNotNull(result);
    Assert.IsInstanceOf<NotFoundObjectResult>(result);
    Assert.IsNotNull(result.Value);
    Assert.IsInstanceOf<String>(result.Value);
    Assert.AreEqual("Shipment Not Found!", result.Value);
}
}
}

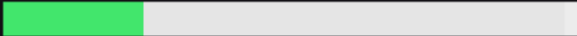
```

Jest Testing

Initial Run

```
RUNS src/App.test.js
RUNS src/tests/AdminDashboard.test.js
RUNS src/tests/UserDashboard.test.js
```

```
Test Suites: 0 of 3 total
Tests:       0 total
Snapshots:   0 total
Time:        1 s, estimated 4 s
```



Snapshot Summary

› 4 snapshots written from 1 test suite.

```
Test Suites: 3 passed, 3 total
Tests:       25 passed, 25 total
Snapshots:   4 written, 4 total
Time:        14.442 s
```

Ran all test suites.

Watch Usage

- › Press **f** to run only failed tests.
- › Press **o** to only run tests related to changed files.
- › Press **q** to quit watch mode.
- › Press **p** to filter by a filename regex pattern.
- › Press **t** to filter by a test name regex pattern.
- › Press **Enter** to trigger a test run.

Initial Snapshots

App.test.js.snap

```
// Jest Snapshot v1, https://goo.gl/fbAQLP

exports[`App Routing matches snapshot for AdminDashboard 1`] = `
<DocumentFragment>
  <div>
    Admin Dashboard
  </div>
</DocumentFragment>
`;

exports[`App Routing matches snapshot for Login page 1`] = `
<DocumentFragment>
  <div>
    Login
  </div>
</DocumentFragment>
`;

exports[`App Routing matches snapshot for Register page 1`] = `
<DocumentFragment>
  <div>
    Register
  </div>
</DocumentFragment>
`;

exports[`App Routing matches snapshot for UserDashboard 1`] = `
<DocumentFragment>
  <div>
    User Dashboard
  </div>
</DocumentFragment>
`;
```

Snapshot Updates

Initial

```
// Mock components
jest.mock('./components/Login', () => () => <div>Login</div>);
jest.mock('./components/Register', () => () => <div>Register</div>);
jest.mock('./components/AdminDashboard', () => () => <div>Admin Dashboard</div>);
jest.mock('./components/UserDashboard', () => () => <div>User Dashboard</div>);
```

Update

```
// Mock components
jest.mock('./components/Login', () => () => <div>Login Page</div>);
jest.mock('./components/Register', () => () => <div>Register Page</div>);
jest.mock('./components/AdminDashboard', () => () => <div>Admin Dashboard</div>);
jest.mock('./components/UserDashboard', () => () => <div>User Dashboard</div>);
```

Diff of the mismatch to the initial snapshot

```
> 2 snapshots failed.  
Snapshot Summary  
> 2 snapshots failed from 1 test suite. Inspect your code changes or press `u` to update them.
```

• App Routing > matches snapshot for Login page

```
expect(received).toMatchSnapshot()
```

```
Snapshot name: `App Routing matches snapshot for Login page 1`
```

```
- Snapshot - 1  
+ Received + 1
```

```
<DocumentFragment>  
  <div>  
-   Login  
+   Login Page  
  </div>  
</DocumentFragment>
```

```
Snapshot name: `App Routing matches snapshot for Register page 1`
```

```
- Snapshot - 1  
+ Received + 1
```

```
<DocumentFragment>  
  <div>  
-   Register  
+   Register Page  
  </div>  
</DocumentFragment>
```

Snapshot Updating

```
PASS src/tests/UserDashboard.test.js
PASS src/tests/AdminDashboard.test.js
PASS src/App.test.js
  > 2 snapshots updated.

Snapshot Summary
  > 2 snapshots updated from 1 test suite.

Test Suites: 3 passed, 3 total
Tests:       25 passed, 25 total
Snapshots:   2 updated, 2 passed, 4 total
Time:        6.733 s
Ran all test suites.

Watch Usage: Press w to show more.
```

Reflection in the snapshot

```
exports[`App Routing matches snapshot for Login page 1`] = `
<DocumentFragment>
  <div>
    Login Page
  </div>
</DocumentFragment>
`;

exports[`App Routing matches snapshot for Register page 1`] = `
<DocumentFragment>
  <div>
    Register Page
  </div>
</DocumentFragment>
`;
```


App.test.js

Project name :	Online Courier Management System
Module Name :	App.jsx
Reference Document :	App.jsx / App.test.js
Created by :	Nisanth Saravanan
Date of creation :	12-08-2025
Date of review :	

LoginRender	The Login Page is to be rendered at the "/" path.	Login Render Success	Mock the Login Page	No user logged in.	The login page is rendered.	The login page is rendered.	Pass
			Create a constant called renderWithRouter that uses renderWithRouter and user role to render a path. Call in renderWithRouter() with "/" path and "null" for user.	Initial render of the app.	The login mock is rendered.	The login mock is rendered.	
RegisterRender	The Register Page is to be rendered at the "/register" path.	Register Render Success	Mock the Register Page	No user logged in.	The register page is rendered.	The register page is rendered.	Pass
			Create a constant called renderWithRouter that uses renderWithRouter and user role to render a path. Call in renderWithRouter() with "/register" path and "null" for user.	Initial render of the app.	The register mock is rendered.	The register mock is rendered.	
UserRender	The User Page is to be rendered at the "/user" path, with "User" role.	User Render Success	Mock the User Page	Logging in as User.	The user page is rendered.	The user page is rendered.	Pass
			Create a constant called renderWithRouter that uses renderWithRouter and user role to render a path. Call in renderWithRouter() with "/user" path and "User" for user.	Initial render of the app.	The user mock is rendered.	The user mock is rendered.	
AdminRender	The Admin Page is to be rendered at the "/admin" path, with "Admin" role.	Admin Render Success	Mock the Admin Page	Logging in as Admin.	The admin page is rendered.	The admin page is rendered.	Pass
			Create a constant called renderWithRouter that uses renderWithRouter and user role to render a path. Call in renderWithRouter() with "/admin" path and "Admin" for user.	Initial render of the app.	The admin mock is rendered.	The admin mock is rendered.	

```
//App.test.js
```

```
import React from 'react';
import { render, screen } from '@testing-library/react';
import App from './App';
import { AuthContext } from './context/AuthContext';
```

```
// Mock components
```

```
jest.mock('./components/Login', () => () => <div>Login</div>);
jest.mock('./components/Register', () => () => <div>Register</div>);
jest.mock('./components/AdminDashboard', () => () => <div>Admin Dashboard</div>);
jest.mock('./components/UserDashboard', () => () => <div>User Dashboard</div>);
```

```
describe('App Routing', () => {
  const renderWithRouter = (initialRoute, user) => {
    window.history.pushState({}, 'Test page', initialRoute);
```

```
return render(  
  <AuthContext.Provider value={{ user }}>  
    <App />  
  </AuthContext.Provider>  
);  
};  
  
it('renders Login page at root route', () => {  
  renderWithRouter('/', null);  
  expect(screen.getByText(/Login/i)).toBeInTheDocument();  
});  
  
it('renders Register page at /register', () => {  
  renderWithRouter('/register', null);  
  expect(screen.getByText(/Register/i)).toBeInTheDocument();  
});  
  
it('renders AdminDashboard for admin user', () => {  
  renderWithRouter('/admin', { role: 'Admin' });  
  expect(screen.getByText(/Admin Dashboard/i)).toBeInTheDocument();  
});  
  
it('redirects to Login if non-admin user accesses /admin', () => {  
  renderWithRouter('/admin', { role: 'User' });  
  expect(screen.getByText(/Login/i)).toBeInTheDocument();  
});  
  
it('renders UserDashboard for user role', () => {  
  renderWithRouter('/user', { role: 'User' });  
  expect(screen.getByText(/User Dashboard/i)).toBeInTheDocument();  
});  
  
it('redirects to Login if non-user accesses /user', () => {  
  renderWithRouter('/user', { role: 'Admin' });  
  expect(screen.getByText(/Login/i)).toBeInTheDocument();  
});  
  
it('matches snapshot for Login page', () => {  
  const { asFragment } = renderWithRouter('/', null);  
  expect(asFragment()).toMatchSnapshot();  
});  
  
it('matches snapshot for Register page', () => {  
  const { asFragment } = renderWithRouter('/register', null);  
  expect(asFragment()).toMatchSnapshot();  
});  
  
it('matches snapshot for AdminDashboard', () => {  
  const { asFragment } = renderWithRouter('/admin', { role: 'Admin' });  
  expect(asFragment()).toMatchSnapshot();  
});
```

```
it('matches snapshot for UserDashboard', () => {  
  const { asFragment } = renderWithRouter('/user', { role: 'User' });  
  expect(asFragment()).toMatchSnapshot();  
});  
});
```

UserDashboard.test.js

Project name :	Online Courier Management System
Module Name :	UserDashboard.jsx
Reference Document :	UserDashboard.jsx / UserDashboard.test.js
Created by :	Nisanth Saravanan
Date of creation :	12-08-2025
Date of review :	

InitialRender	Renders dashboard and fetches shipments	Render and Show Shipments	Render the User Dashboard page.	Mock the api.	See the "User Dashboard" heading on screen.	See the "User Dashboard" heading on screen.	Pass
				Mock user with role "User".	See the mockdata on screen.	See the mockdata on screen.	
				Mock some Data.			
				Mock the get response with mockdata.			
				Logged in as User.			
FilterBy001	Filters shipments based on name	Filter by name success	Render the User Dashboard page.	Mock the api.	The mockdata with matching name remains on screen.	The mockdata with matching name remains on screen.	Pass
			Wait for Mockdata to appear.	Mock user with role "User".	Rest of the mock data is not on the screen.	Rest of the mock data is not on the screen.	
			Target the searchbar and send keys of a mockdata's name.	Mock some Data.			
				Mock the get response with mockdata.			
				Logged in as User.			
FilterBy002	Filters shipments based on type	Filter by type success	Render the User Dashboard page.	Mock the api.	The mockdata with matching type remains on screen.	The mockdata with matching type remains on screen.	Pass
			Wait for Mockdata to appear.	Mock user with role "User".	Rest of the mock data is not on the screen.	Rest of the mock data is not on the screen.	
			Target the searchbar and send keys of a mockdata's type.	Mock some Data.			
				Mock the get response with mockdata.			
				Logged in as User.			
FilterBy003	Filters shipments based on address	Filter by address success	Render the User Dashboard page.	Mock the api.	The mockdata with matching address remains on screen.	The mockdata with matching address remains on screen.	Pass
			Wait for Mockdata to appear.	Mock user with role "User".	Rest of the mock data is not on the screen.	Rest of the mock data is not on the screen.	
			Target the searchbar and send keys of a mockdata's address.	Mock some Data.			
				Mock the get response with mockdata.			
				Logged in as User.			
LogoutUser	Calls logout and clears user.	Logout Success	Render the User Dashboard page.	Mock user with role "User".	Redirected to Login Page.	Redirected to Login Page.	Pass
			Target the Logout Button and fire click event.	Logged in as User.			

//User test

```
import React from 'react';
import { render, screen, fireEvent, waitFor, act } from '@testing-library/react';
import { AuthContext } from '../context/AuthContext';
import { MemoryRouter } from 'react-router-dom';
import UserDashboard from '../components/UserDashboard';
import API from '../services/api';
```

```
jest.mock('../services/api');
```

```
const mockUser = { role: 'user' };
const mockSetUser = jest.fn();
```

```
const mockShipments = [
```

```

{ id: 1, name: 'Box A', type: 'Electronics', address: '123 Street' },
{ id: 2, name: 'Box B', type: 'Clothing', address: '456 Avenue' },
];

describe('UserDashboard', () => {
  beforeEach(() => {
    API.get.mockResolvedValue({ data: mockShipments });
  });

  it('renders dashboard and fetches shipments', async () => {
    await act(async () => {
      render(
        <MemoryRouter>
          <AuthContext.Provider value={{ user: mockUser, setUser: mockSetUser }}>
            <UserDashboard />
          </AuthContext.Provider>
        </MemoryRouter>
      );
    });

    expect(screen.getByText(/User Dashboard/i)).toBeInTheDocument();

    await waitFor(() => {
      expect(screen.getByText(/Box A/i)).toBeInTheDocument();
      expect(screen.getByText(/Box B/i)).toBeInTheDocument();
    });

    it('filters shipments based on name', async () => {
      await act(async () => {
        render(
          <MemoryRouter>
            <AuthContext.Provider value={{ user: mockUser, setUser: mockSetUser }}>
              <UserDashboard />
            </AuthContext.Provider>
          </MemoryRouter>
        );
      });

      await waitFor(() => screen.getByText(/Box A/i));

      act(() => {
        fireEvent.change(screen.getByPlaceholderText(/Search by name/i), {
          target: { value: 'Box A' },
        });
      });

      expect(screen.queryByText(/Box B/i)).not.toBeInTheDocument();
      expect(screen.getByText(/Box A/i)).toBeInTheDocument();
    });
  });
});

```

```
it('filters shipments based on type', async () => {
  await act(async () => {
    render(
      <MemoryRouter>
        <AuthContext.Provider value={{ user: mockUser, setUser: mockSetUser }}>
          <UserDashboard />
        </AuthContext.Provider>
      </MemoryRouter>
    );
  });
});
```

```
await waitFor(() => screen.getByText(/Box A/i));
```

```
act(() => {
  fireEvent.change(screen.getByPlaceholderText(/Search by name/i), {
    target: { value: 'Electronics' },
  });
});
```

```
expect(screen.queryByText(/Box B/i)).not.toBeInTheDocument();
expect(screen.getByText(/Box A/i)).toBeInTheDocument();
});
```

```
it('filters shipments based on address', async () => {
  await act(async () => {
    render(
      <MemoryRouter>
        <AuthContext.Provider value={{ user: mockUser, setUser: mockSetUser }}>
          <UserDashboard />
        </AuthContext.Provider>
      </MemoryRouter>
    );
  });
});
```

```
await waitFor(() => screen.getByText(/Box A/i));
```

```
act(() => {
  fireEvent.change(screen.getByPlaceholderText(/Search by name/i), {
    target: { value: '123 Street' },
  });
});
```

```
expect(screen.queryByText(/Box B/i)).not.toBeInTheDocument();
expect(screen.getByText(/Box A/i)).toBeInTheDocument();
});
```

```
it('calls logout and clears user', async () => {
  await act(async () => {
    render(
      <MemoryRouter>
        <AuthContext.Provider value={{ user: mockUser, setUser: mockSetUser }}>
```

```
        <UserDashboard />
      </AuthContext.Provider>
    </MemoryRouter>
  );
});
```

```
act(() => {
  fireEvent.click(screen.getByText(/Logout/i));
});
```

```
expect(mockSetUser).toHaveBeenCalledWith(null);
});
});
```

AdminDashboard.test.js

Project name :	Online Courier Management System
Module Name :	AdminDashboard.jsx
Reference Document :	AdminDashboard.jsx / AdminDashboard.test.js
Created by :	Nisanth Saravanan
Date of creation :	12-08-2025
Date of review :	

InitialRender	Renders dashboard and fetches shipments	Render and Show Shipments	Render the Admin Dashboard page.	Mock the api.	See the "Admin Dashboard" heading on screen.	See the "Admin Dashboard" heading on screen.	Pass
				Mock user with role "Admin".	See the mockdata on screen.	See the mockdata on screen.	
				Mock some Data.			
				Mock the get response with mockdata.			
				Logged in as Admin.			
FilterBy001	Filters shipments based on name	Filter by name success	Render the Admin Dashboard page.	Mock the api.	The mockdata with matching name remains on screen.	The mockdata with matching name remains on screen.	Pass
			Wait for Mockdata to appear.	Mock user with role "Admin".	Rest of the mock data is not on the screen.	Rest of the mock data is not on the screen.	
			Target the searchbar and send keys of a mockdata's name.	Mock some Data.			
				Mock the get response with mockdata.			
				Logged in as Admin.			
FilterBy002	Filters shipments based on type	Filter by type success	Render the Admin Dashboard page.	Mock the api.	The mockdata with matching type remains on screen.	The mockdata with matching type remains on screen.	Pass
			Wait for Mockdata to appear.	Mock user with role "Admin".	Rest of the mock data is not on the screen.	Rest of the mock data is not on the screen.	
			Target the searchbar and send keys of a mockdata's type.	Mock some Data.			
				Mock the get response with mockdata.			
				Logged in as Admin.			
FilterBy003	Filters shipments based on address	Filter by address success	Render the Admin Dashboard page.	Mock the api.	The mockdata with matching address remains on screen.	The mockdata with matching address remains on screen.	Pass
			Wait for Mockdata to appear.	Mock user with role "Admin".	Rest of the mock data is not on the screen.	Rest of the mock data is not on the screen.	
			Target the searchbar and send keys of a mockdata's address.	Mock some Data.			
				Mock the get response with mockdata.			
				Logged in as Admin.			
LogoutAdmin	Calls logout and clears user.	Logout Success	Render the Admin Dashboard page.	Mock user with role "Admin".	Redirected to Login Page.	Redirected to Login Page.	Pass
			Target the Logout Button and fire click event.	Logged in as Admin.			
FormFieldUpdates001	Updates form fields correctly	Fields Updates Success	Render the Admin Dashboard page.	Mock user with role "Admin".	The input fields are duly updated with the given data.	The input fields are duly updated with the given data.	Pass
			Target the Name input field and send keys.	Logged in as Admin.			
			Target the Type input field and send keys.				
			Target the Address input field and send keys.				
AddShipment	Adds a shipment when form is complete	Add Shipment Success	Render the Admin Dashboard page.	Mock the api.	The name of the new shipment is in the document.	The name of the new shipment is in the document.	Pass
			Target the Name input field and send keys.	Mock user with role "Admin".			
			Target the Type input field and send keys.	Mock some Data.			
			Target the Address input field and send keys.	Mock the get response with mockdata.			
			Target the Submit Button and fire the click event.	Logged in as Admin.			
				Mock post response with new shipment data.			
FormFieldUpdates002	Populates form when editing a shipment	Form Field Populated	Render the Admin Dashboard page.	Mock the api.	The form fields are populated with the data of the selected shipment.	The form fields are populated with the data of the selected shipment.	Pass
			Target an Edit Button and fire a click event.	Mock user with role "Admin".			
				Mock some Data.			
				Mock the get response with mockdata.			
				Logged in as Admin.			

ShipmentDelete	Deletes a shipment when prompted to.	Deletion Success	Render the Admin Dashboard page.	Mock the api.	The delete shipment is nowhere to be seen on the screen.	The delete shipment is nowhere to be seen on the screen.	Pass
			Target any Delete Button and fire a click event.	Mock user with role "Admin".			
				Mock some Data.			
				Mock the get response with mock data.			
				Logged in as Admin.			
AlertAdmin	Shows alert when form is submitted in complete	Alerted for an empty form submission.	Mock delete response with new shipment data.				
			Render the Admin Dashboard page.	Mock user with role "Admin".	An alert is to be expected on the window with a particular pre-coded message.	An alert is to be expected on the window with a particular pre-coded message.	Pass
			Target the Submit Button and fire a click event.	Logged in as Admin.			

//Admin Test

```
import React from 'react';
import { render, screen, fireEvent, waitFor, act } from '@testing-library/react';
import { AuthContext } from '../context/AuthContext';
import { MemoryRouter } from 'react-router-dom';
import AdminDashboard from '../components/AdminDashboard';
import API from '../services/api';
```

```
jest.mock('../services/api');
```

```
const mockUser = { role: 'admin' };
const mockSetUser = jest.fn();
```

```
const mockShipments = [
  { id: 1, name: 'Box A', type: 'Electronics', address: '123 Street' },
  { id: 2, name: 'Box B', type: 'Clothing', address: '456 Avenue' },
];
```

```
describe('AdminDashboard', () => {
  beforeEach(() => {
    API.get.mockResolvedValue({ data: mockShipments });
  });
```

```
it('renders dashboard and fetches shipments', async () => {
  await act(async () => {
    render(
      <MemoryRouter>
        <AuthContext.Provider value={{ user: mockUser, setUser: mockSetUser }}>
          <AdminDashboard />
        </AuthContext.Provider>
      </MemoryRouter>
    );
  });
```

```
expect(screen.getByText(/Admin Dashboard/i)).toBeInTheDocument();
```

```
await waitFor(() => {
  expect(screen.getByText(/Box A/i)).toBeInTheDocument();
```

```

    expect(screen.getByText(/Box B/i)).toBeInTheDocument();
  });
});

it('filters shipments based on name', async () => {
  await act(async () => {
    render(
      <MemoryRouter>
        <AuthContext.Provider value={{ user: mockUser, setUser: mockSetUser }}>
          <AdminDashboard />
        </AuthContext.Provider>
      </MemoryRouter>
    );
  });

  await waitFor(() => screen.getByText(/Box A/i));

  act(() => {
    fireEvent.change(screen.getByPlaceholderText(/Search by name/i), {
      target: { value: 'Box A' },
    });
  });

  expect(screen.queryByText(/Box B/i)).not.toBeInTheDocument();
  expect(screen.getByText(/Box A/i)).toBeInTheDocument();
});

it('filters shipments based on type', async () => {
  await act(async () => {
    render(
      <MemoryRouter>
        <AuthContext.Provider value={{ user: mockUser, setUser: mockSetUser }}>
          <AdminDashboard />
        </AuthContext.Provider>
      </MemoryRouter>
    );
  });

  await waitFor(() => screen.getByText(/Box A/i));

  act(() => {
    fireEvent.change(screen.getByPlaceholderText(/Search by name/i), {
      target: { value: 'Electronics' },
    });
  });

  expect(screen.queryByText(/Box B/i)).not.toBeInTheDocument();
  expect(screen.getByText(/Box A/i)).toBeInTheDocument();
});

it('filters shipments based on address', async () => {

```

```

await act(async () => {
  render(
    <MemoryRouter>
      <AuthContext.Provider value={{ user: mockUser, setUser: mockSetUser }}>
        <AdminDashboard />
      </AuthContext.Provider>
    </MemoryRouter>
  );
});

```

```

await waitFor(() => screen.getByText(/Box A/i));

```

```

act(() => {
  fireEvent.change(screen.getByPlaceholderText(/Search by name/i), {
    target: { value: '123 Street' },
  });
});

```

```

expect(screen.queryByText(/Box B/i)).not.toBeInTheDocument();
expect(screen.getByText(/Box A/i)).toBeInTheDocument();
});

```

```

it('calls logout and clears user', async () => {
  await act(async () => {
    render(
      <MemoryRouter>
        <AuthContext.Provider value={{ user: mockUser, setUser: mockSetUser }}>
          <AdminDashboard />
        </AuthContext.Provider>
      </MemoryRouter>
    );
  });
});

```

```

act(() => {
  fireEvent.click(screen.getByText(/Logout/i));
});

```

```

expect(mockSetUser).toHaveBeenCalledWith(null);
});

```

```

it('updates form fields correctly', async () => {
  await act(async () => {
    render(
      <MemoryRouter>
        <AuthContext.Provider value={{ user: mockUser, setUser: mockSetUser }}>
          <AdminDashboard />
        </AuthContext.Provider>
      </MemoryRouter>
    );
  });
});

```

```

fireEvent.change(screen.getByPlaceholderText('Name'), {
  target: { value: 'New Shipment' },
});
fireEvent.change(screen.getByPlaceholderText('Type'), {
  target: { value: 'Books' },
});
fireEvent.change(screen.getByPlaceholderText('Address'), {
  target: { value: '789 Boulevard' },
});

expect(screen.getByPlaceholderText('Name').value).toBe('New Shipment');
expect(screen.getByPlaceholderText('Type').value).toBe('Books');
expect(screen.getByPlaceholderText('Address').value).toBe('789 Boulevard');
});

it('adds a shipment when form is complete', async () => {
  const newShipment = { id: 3, name: 'Box C', type: 'Books', address: '789
Boulevard' };

  // Initial fetch: Box A and Box B
  API.get.mockResolvedValueOnce({ data: mockShipments });

  // Mock POST response
  API.post.mockResolvedValue({ data: newShipment });

  // After adding: Box A, Box B, and Box C
  API.get.mockResolvedValueOnce({ data: [...mockShipments, newShipment] });

  await act(async () => {
    render(
      <MemoryRouter>
        <AuthContext.Provider value={{ user: mockUser, setUser: mockSetUser }}>
          <AdminDashboard />
        </AuthContext.Provider>
      </MemoryRouter>
    );
  });

  // Fill form
  fireEvent.change(screen.getByPlaceholderText('Name'), {
    target: { value: 'Box C' },
  });
  fireEvent.change(screen.getByPlaceholderText('Type'), {
    target: { value: 'Books' },
  });
  fireEvent.change(screen.getByPlaceholderText('Address'), {
    target: { value: '789 Boulevard' },
  });

  // Submit form
  await act(async () => {

```

```

    fireEvent.click(screen.getByText(/Add Shipment/i));
  });

  // Wait for re-render with new shipment
  await waitFor(() => {
    expect(screen.getByText(/Box C/i)).toBeInTheDocument();
  });
});

it('populates form when editing a shipment', async () => {
  await act(async () => {
    render(
      <MemoryRouter>
        <AuthContext.Provider value={{ user: mockUser, setUser: mockSetUser }}>
          <AdminDashboard />
        </AuthContext.Provider>
      </MemoryRouter>
    );
  });

  await waitFor(() => screen.getByText(/Box A/i));

  fireEvent.click(screen.getAllByText(/Edit/i)[0]);

  expect(screen.getByPlaceholderText('Name').value).toBe('Box A');
  expect(screen.getByPlaceholderText('Type').value).toBe('Electronics');
  expect(screen.getByPlaceholderText('Address').value).toBe('123 Street');
});

it('deletes a shipment', async () => {
  // Initial mock: both shipments
  API.get.mockResolvedValueOnce({ data: mockShipments });

  // Mock delete response
  API.delete.mockResolvedValue({});

  // After deletion: only Box B remains
  API.get.mockResolvedValueOnce({ data: [mockShipments[1]] });

  await act(async () => {
    render(
      <MemoryRouter>
        <AuthContext.Provider value={{ user: mockUser, setUser: mockSetUser }}>
          <AdminDashboard />
        </AuthContext.Provider>
      </MemoryRouter>
    );
  });

  // Wait for initial render
  await waitFor(() => {

```

```

    expect(screen.getByText(/Box A/i)).toBeInTheDocument();
  });

  // Trigger delete
  await act(async () => {
    fireEvent.click(screen.getAllByText(/Delete/i)[0]);
  });

  // Wait for re-render after deletion
  await waitFor(() => {
    expect(screen.queryByText(/Box A/i)).not.toBeInTheDocument();
    expect(screen.getByText(/Box B/i)).toBeInTheDocument();
  });
});

```

```

it('shows alert when form is incomplete', async () => {
  window.alert = jest.fn();

```

```

  await act(async () => {
    render(
      <MemoryRouter>
        <AuthContext.Provider value={{ user: mockUser, setUser: mockSetUser }}>
          <AdminDashboard />
        </AuthContext.Provider>
      </MemoryRouter>
    );
  });

```

```

  act(() => {
    fireEvent.click(screen.getByText(/Add Shipment/i));
  });

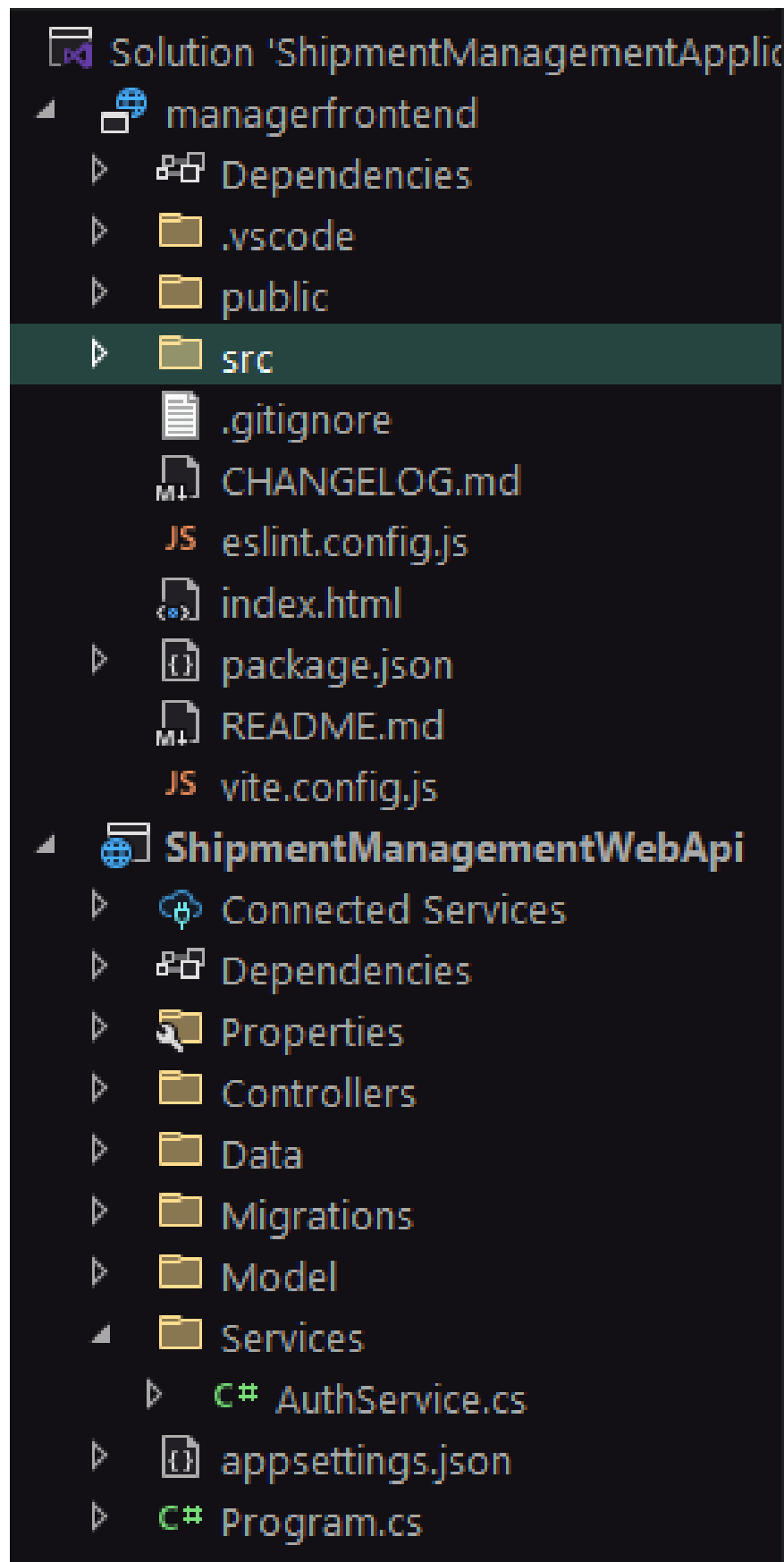
```

```

  expect(window.alert).toHaveBeenCalledWith('All fields Needed!');
});
});

```

Code Base



Backend:

/Services/AuthService.cs

```
using ShipmentManagementWebApi.Data;
using ShipmentManagementWebApi.Models;
using Microsoft.EntityFrameworkCore;

namespace ShipmentManagementWebApi.Services;

public class AuthService
{
    private readonly AppDbContext _context;

    public AuthService(AppDbContext context)
    {
        _context = context;
    }

    public User Authenticate(string username, string password)
    {
        return _context.Users.FirstOrDefault(u => u.Username == username && u.Password
== password);
    }

    public bool Register(User user)
    {
        if (_context.Users.Any(u => u.Username == user.Username)) return false;
        _context.Users.Add(user);
        _context.SaveChanges();
        return true;
    }
}
```


appsettings.json

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "ConnectionStrings": {
    "DefaultConnection":
"Server=127.0.0.1;Port=3306;Database=ShipmentDb;User=root;Password=root"
  },
  "AllowedHosts": "*"
}
```

Program.cs

```
using Microsoft.EntityFrameworkCore;
using ShipmentManagementWebApi.Data;
using ShipmentManagementWebApi.Services;

var builder = WebApplication.CreateBuilder(args);

// Add services to the container.

builder.Services.AddControllers();
// Learn more about configuring Swagger/OpenAPI at
https://aka.ms/aspnetcore/swashbuckle
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

// Configure EF Core with MySQL
builder.Services.AddDbContext<AppDbContext>(options =>
    options.UseMySQL(
        builder.Configuration.GetConnectionString("DefaultConnection"),

        ServerVersion.AutoDetect(builder.Configuration.GetConnectionString("DefaultConnection")
    ));

// Register custom services
builder.Services.AddScoped<AuthService>();

builder.Services.AddCors(options =>
{
    options.AddPolicy("AllowReactApp",
        policy => policy
            .WithOrigins("http://localhost:51097") // React dev server
            .AllowAnyHeader()
            .AllowAnyMethod());
});

var app = builder.Build();

app.UseCors("AllowReactApp");

// Configure the HTTP request pipeline.
if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}

app.UseHttpsRedirection();

app.UseAuthorization();

app.MapControllers();

app.Run();
```

/Models/User.cs

```
using System.ComponentModel.DataAnnotations;

namespace ShipmentManagementWebApi.Models
{
    public class User
    {
        [Key]
        public int Id { get; set; }
        public string? Username { get; set; }
        public string? Password { get; set; } // Plain text for simplicity
        public string? Role { get; set; } = "User"; // Default role
    }
}
```

/Models/Shipment.cs

```
using System.ComponentModel.DataAnnotations;

namespace ShipmentManagementWebApi.Models
{
    public class Shipment
    {
        [Key]
        public int Id { get; set; }
        public string? Name { get; set; }
        public string? Type { get; set; }
        public string? Address { get; set; }
    }
}
```

/Data/AppDbContext.cs

```
using Microsoft.EntityFrameworkCore;
using System.Collections.Generic;
using ShipmentManagementWebApi.Models;

namespace ShipmentManagementWebApi.Data
{
    public class AppDbContext : DbContext
    {
        public AppDbContext(DbContextOptions<AppDbContext> options) : base(options)
        { }

        public DbSet<User> Users { get; set; }
        public DbSet<Shipment> Shipments { get; set; }
    }
}
```

/Controllers/AuthController.cs

```
using Microsoft.AspNetCore.Mvc;
using ShipmentManagementWebApi.Services;
using ShipmentManagementWebApi.Models;

[ApiController]
[Route("api/[controller]")]
public class AuthController : ControllerBase
{
    private readonly AuthService _authService;

    public AuthController(AuthService authService)
    {
        _authService = authService;
    }

    [HttpPost("register")]
    public IActionResult Register(User user)
    {
        user.Role = "User"; // Default role
        if (_authService.Register(user)) return Ok("Registered successfully");
        return BadRequest("Username already exists");
    }

    [HttpPost("login")]
    public IActionResult Login(User user)
    {
        var loggedInUser = _authService.Authenticate(user.Username, user.Password);
        if (loggedInUser == null) return Unauthorized("Invalid credentials");
        return Ok(loggedInUser); // Return user object with role
    }
}
```

/Controllers/ShipmentController.cs

```
using Microsoft.AspNetCore.Mvc;
using ShipmentManagementWebApi.Data;
using ShipmentManagementWebApi.Models;

[ApiController]
[Route("api/[controller]")]
public class ShipmentController : ControllerBase
{
    private readonly ApplicationDbContext _context;

    public ShipmentController(ApplicationDbContext context)
    {
        _context = context;
    }

    [HttpGet]
    public IActionResult GetAll() => Ok(_context.Shipments.ToList());

    [HttpGet("{id}")]
    public IActionResult Get(int id)
    {
        var shipment = _context.Shipments.Find(id);
        if (shipment == null) return NotFound();
        return Ok(shipment);
    }

    [HttpPost]
    public IActionResult Create(Shipment shipment, [FromQuery] string role)
    {
        if (role != "Admin") return Unauthorized("Only admin can create shipments");
        _context.Shipments.Add(shipment);
        _context.SaveChanges();
        return Ok(shipment);
    }

    [HttpPut("{id}")]
    public IActionResult Update(int id, Shipment updatedshipment, [FromQuery] string
role)
    {
        if (role != "Admin") return Unauthorized("Only admin can update shipments");
        var shipment = _context.Shipments.Find(id);
        if (shipment == null) return NotFound();

        shipment.Name = updatedshipment.Name;
        shipment.Type = updatedshipment.Type;
        shipment.Address = updatedshipment.Address;

        _context.SaveChanges();
        return Ok(shipment);
    }

    [HttpDelete("{id}")]
    public IActionResult Delete(int id, [FromQuery] string role)
    {
        if (role != "Admin") return Unauthorized("Only admin can delete shipments");
        var shipment = _context.Shipments.Find(id);
        if (shipment == null) return NotFound();

        _context.Shipments.Remove(shipment);
        _context.SaveChanges();
    }
}
```

```
        return Ok("Deleted");  
    }  
}
```


Frontend – React JS

/src/main.jsx

```
import React from 'react';
import ReactDOM from 'react-dom/client';
import App from './App';
import { AuthProvider } from './context/AuthContext';
import './index.css';

const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(
  <AuthProvider>
    <App />
  </AuthProvider>
);
```

/src/App.jsx

```
import React, { useContext } from 'react';
import { BrowserRouter as Router, Routes, Route, Navigate } from 'react-router-dom';
import Login from './components/Login';
import Register from './components/Register';
import AdminDashboard from './components/AdminDashboard';
import UserDashboard from './components/UserDashboard';
import { AuthContext } from './context/AuthContext';
import './App.css';

function App() {
  const { user } = useContext(AuthContext);

  return (
    <Router>
      <Routes>
        <Route path="/" element={<Login />} />
        <Route path="/register" element={<Register />} />
        <Route path="/admin" element={user?.role === 'Admin' ? <AdminDashboard
/> : <Navigate to="/" />} />
        <Route path="/user" element={user?.role === 'User' ? <UserDashboard />
: <Navigate to="/" />} />
      </Routes>
    </Router>
  );
}

export default App;
```

/services/api.js

```
//api.js
import axios from 'axios';

const API = axios.create({
  baseURL: 'http://localhost:5099/api', // Adjust if needed
});

export default API;
```

/context/AuthContext.jsx

```
//AuthContext.jsx
import React, { createContext, useState } from 'react';

// eslint-disable-next-line react-refresh/only-export-components
export const AuthContext = createContext();

export const AuthProvider = ({ children }) => {
  const [user, setUser] = useState(null); // Holds logged-in user

  return (
    <AuthContext.Provider value={{ user, setUser }}>
      {children}
    </AuthContext.Provider>
  );
};
```

/components/Login.jsx

```
import React, { useState, useContext, useEffect } from 'react';
import { useNavigate } from 'react-router-dom';
import API from '../services/api';
import { AuthContext } from '../context/AuthContext';

function Login() {
  const [username, setUsername] = useState('');
  const [password, setPassword] = useState('');
  const { setUser } = useContext(AuthContext);
  const { user } = useContext(AuthContext);
  const navigate = useNavigate();

  const handleLogin = async () => {
    if (username == '' || password == '') {
      alert("All credentials are mandatory.");
      return;
    }

    try {
      const res = await API.post('/auth/login', { username, password });
      setUser(res.data);
      res.data.role === 'Admin' ? navigate('/admin') : navigate('/user');
    } catch {
      alert('Invalid credentials');
    }
  };

  const handleRegister = () => {
    navigate('/register');
  };

  useEffect(() => {
    document.title = "Login";

    if (user?.role == "Admin") {
      navigate("/admin");
    }
    else if (user?.role == "User") {
      navigate("/user")
    }
  });

  return (
    <>
      <div class="container">
        <h2>Login</h2>
        <table>
          <tr>
            <td>
              <div class="input-group input-group-lg mb-3">
                <input class="form-control" placeholder="Username"
onChange={e => setUsername(e.target.value)} />
              </div>
            </td>
          </tr>
          <tr>
            <td>
              <div class="input-group input-group-lg mb-3">
                <input class="form-control" placeholder="Password"
type="password" onChange={e => setPassword(e.target.value)} />
              </div>
            </td>
          </tr>
        </table>
      </div>
    </>
  );
}
```

```

        </div>
      </td>
    </tr>
    <tr>
      <td>
        <button onClick={handleLogin}>Login</button>
      </td>
    </tr>
    <br><br>
    <tr>
      <td>
        <button onClick={handleRegister}>New User?
Register.</button>
      </td>
    </tr>
  </table>
</div>
</>
);
}

export default Login;

```

/components/Register.jsx

```
import React, { useState, useEffect, useContext } from 'react';
import { useNavigate } from 'react-router-dom';
import API from '../services/api';
import { AuthContext } from '../context/AuthContext';

function Register() {
  const [username, setUsername] = useState('');
  const [password, setPassword] = useState('');
  const { user } = useContext(AuthContext);
  const navigate = useNavigate();

  const handleRegister = async () => {
    if (username == '' || password == '') {
      alert("All credentials are mandatory.");
      return;
    }

    try {
      await API.post('/auth/register', { username, password });
      alert('Registration successful');
      navigate("/");
    } catch {
      alert('Registration failed');
    }
  };

  const handleLogin = () => {
    navigate('/');
  };

  useEffect(() => {
    document.title = "Register";

    if (user?.role == "Admin") {
      navigate("/admin");
    }
    else if (user?.role == "User") {
      navigate("/user")
    }
  });

  return (
    <>
      <div class="container">
        <h2>Register</h2>
        <table>
          <tr>
            <td>
              <div class="input-group input-group-lg mb-3">
                <input class="form-control" placeholder="Username"
onChange={e => setUsername(e.target.value)} />
              </div>
            </td>
          </tr>
          <tr>
            <td>
              <div class="input-group input-group-lg mb-3">
                <input class="form-control" placeholder="Password"
type="password" onChange={e => setPassword(e.target.value)} />
              </div>
            </td>
          </tr>
        </table>
      </div>
    </>
  );
}
```

```
        </td>
      </tr>
      <tr>
        <td>
          <button onClick={handleRegister}>Register</button>
        </td>
      </tr>
      <br><br>
      <tr>
        <td>
          <button onClick={handleLogin}>Old User? Login.</button>
        </td>
      </tr>
    </table>
  </div>
</>
  );
}

export default Register;
```


/components/AdminDashboard.jsx

```
import React, { useContext, useEffect, useState } from 'react';
import API from '../services/api';
import { AuthContext } from '../context/AuthContext';

function AdminDashboard() {
  const { user } = useContext(AuthContext);
  const [shipments, setShipment] = useState([]);
  const { setUser } = useContext(AuthContext);
  const [searchTerm, setSearchTerm] = useState('');
  const [form, setForm] = useState({ name: '', type: '', address: '' });
  const [editingId, setEditingId] = useState(null);

  const fetchShipments = async () => {
    const res = await API.get('/shipment');
    setShipment(res.data);
  };

  const createOrUpdateShipment = async () => {
    if (form.name == '' || form.type == '' || form.address == '') {
      alert("All fields Needed!");
      return;
    }

    if (editingId) {
      await API.put(`/shipment/${editingId}?role=${user.role}`, form);
      setEditingId(null);
    } else {
      await API.post(`/shipment?role=${user.role}`, form);
    }
    setForm({ name: '', type: '', address: '' });
    fetchShipments();
  };

  const deleteShipment = async (id) => {
    await API.delete(`/shipment/${id}?role=${user.role}`);
    fetchShipments();
  };

  const startEdit = (shipment) => {
    setForm(shipment);
    setEditingId(shipment.id);
  };

  const filteredShipments = shipments.filter(s =>
    s.name.toLowerCase().includes(searchTerm.toLowerCase()) ||
    s.type.toLowerCase().includes(searchTerm.toLowerCase()) ||
    s.address.toString().includes(searchTerm)
  );

  const logout = () => {
    setUser(null);
  };

  useEffect(() => {
    document.title = "Admin Dash";

    fetchShipments();
  }, []);
}
```

[illegible]

```
    }}  
    </ul>  
  </div>  
);  
}  
  
export default AdminDashboard;
```

/components/UserDashboard.jsx

```
import React, { useContext, useEffect, useState } from 'react';
import API from '../services/api';
import { AuthContext } from '../context/AuthContext';

function UserDashboard() {
  const [shipments, setShipment] = useState([]);
  const [searchTerm, setSearchTerm] = useState('');
  const { setUser } = useContext(AuthContext);

  useEffect(() => {
    document.title = "User Dash";

    API.get('/shipment').then(res => setShipment(res.data));
  }, []);

  const logout = () => {
    setUser(null);
  };

  const filteredShipments = shipments.filter(s =>
    s.name.toLowerCase().includes(searchTerm.toLowerCase()) ||
    s.type.toLowerCase().includes(searchTerm.toLowerCase()) ||
    s.address.toString().includes(searchTerm)
  );

  return (
    <div class="container">
      <h2>User Dashboard</h2>
      <br></br>
      <button onClick={logout} style={{ marginBottom: '1rem' }}>
        Logout
      </button>
      <br></br>
      <br></br>
      <table>
        <tr>
          <td>
            <input
              class="form-control"
              placeholder="Search by name, type, or address"
              value={searchTerm}
              onChange={e => setSearchTerm(e.target.value)}
            />
          </td>
        </tr>
      </table>
      <br></br>
      <br></br>
      <ul>
        {filteredShipments.map(s => (
          <li key={s.id}>
            <strong>Name:</strong> {s.name},&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&~
            <strong>Type:</strong> {s.type},&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&~
            <strong>Addressed To:</strong> {s.address} &nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&~
          </li>
        ))}
      </ul>
    </div>
  );
}
```

```
export default UserDashboard;
```